KU LEUVEN

Advanced Computer Architecture

Lab Report 5

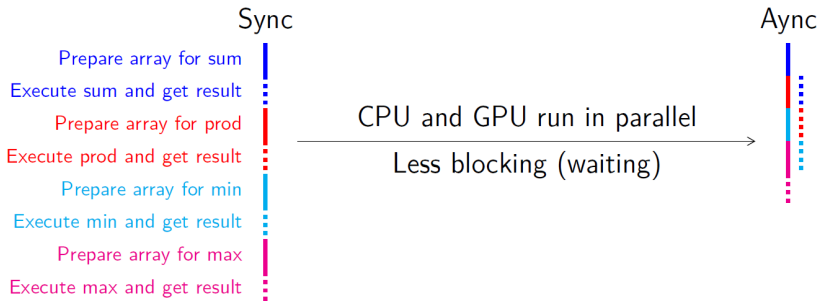
Bert Pyck
Hugo Charmant

ELICT: ELECFS
20 maart 2025
Master in Engineering Technology
Academic year 2024-2025

1.1 Task description

The goal of this report is to compare synchronous and asynchronous executions for four operations: sum, product, min, and max. These computations will be performed on four independent arrays, each initialized within a loop. In the synchronous approach, every execution occurs sequentially, meaning each operation must complete before the next begins. In the asynchronous approach prepares new data while the previous computations are still running on the GPU, improving efficiency.

The task is described in the following figure: 1.1.



Figuur 1.1: Description of the task

1.2 Introduction

We started the lab session by learning about dynamic parallelism and (a)synchronous operations. We learned that dynamic parallelism allows the CPU to launch a GPU kernel (which we already knew) but also enables a GPU kernel to launch another kernel. This is particularly useful in applications like adaptive grid generation (as seen in the slides). Asynchronous operation focuses on optimizing CPU and GPU usage by avoiding blocking during memory transfers and kernel execution.

To demonstrate our understanding, we completed the exercise. First, we executed the operations synchronously (sequentially), and then we executed them asynchronously. This was achieved using **cudaMemcpyAsync()** and **CUDA-streams**, allowing multiple processing tasks to run in parallel.

1.3 Results and conclusion

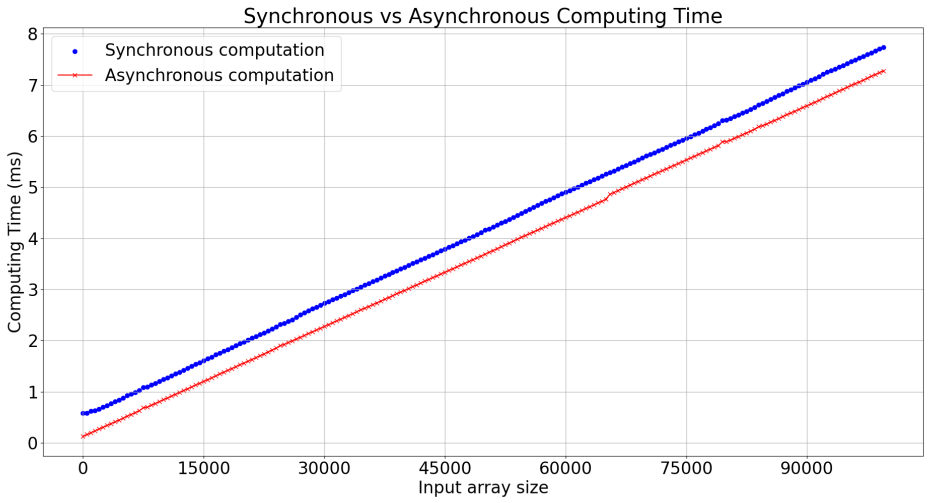
In order to showcase the difference between synchronous and asynchronous computation, we created following test code:

1. Create random array
2. Sum operation
3. Create random array
4. Product operation
5. Create random array
6. Minimum operation
7. Create random array
8. Maximum operation

This was repeated both for the synchronous and asynchronous situation. The details are discussed here

1.3.1 Approach 1: Synchronous executions

For the synchronous execution code, the code didn't change much compared to lab 2. We did add *cudaDeviceSynchronize();* commands after every operation in order to make sure that the GPU operation was completely finished before moving on. This had to be done since the kernel invocation is inherently asynchronous.



Figuur 1.2: Results sync vs async

1.3.2 Approach 2: Asynchronous executions

In the asynchronous execution approach, our initial attempt involved calling *cudaFree();* immediately after every computation. However, since *cudaFree();* waits for the GPU to finish operations before freeing memory, it introduced synchronous behavior. To resolve this, we removed memory deallocation until after timing had been stopped, thereby ensuring that our measurements showed asynchronous behavior.

For the asynchronous memory allocation, the specific function *cudaMemcpyAsync();* was used. By creating four separate CUDA streams—one each for the sum, product, minimum, and maximum operations—we enabled these operations to execute concurrently. This setup allows the GPU to overlap data transfers with kernel computations, effectively utilizing its resources and reducing the overall runtime. The results can be found in figure 1.2. We can clearly see the reduction in computation time when using asynchronous operation.

2 CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include <limits.h>
#include <time.h>

#define THREADS_PER_BLOCK 1024
#define AVERAGE 100

// ---------------------
// Reduction Kernels
// ---------------------

// Maximum reduction kernel
__global__ void max_gpu(unsigned int *d_input, unsigned int *d_output, int size)
    {
    extern __shared__ unsigned int s_data[];
    int tid = threadIdx.x;
    int i = blockIdx.x * (blockDim.x * 2) + tid;
    unsigned int myMax = 0;
    if (i < size)
        myMax = d_input[i];
    if (i + blockDim.x < size)
        myMax = max(myMax, d_input[i + blockDim.x]);
    s_data[tid] = myMax;
    __syncthreads();
    for (unsigned int s = blockDim.x / 2; s > 32; s >>= 1) {
        if (tid < s)
            s_data[tid] = max(s_data[tid], s_data[tid + s]);
        __syncthreads();
    }
    if (tid == 0)
        d_output[blockIdx.x] = s_data[0];
}

// Sum reduction kernel
__global__ void sum_gpu(unsigned int *d_input, unsigned int *d_output, int size)
    {
    extern __shared__ unsigned int s_data[];
    int tid = threadIdx.x;
    int i = blockIdx.x * (blockDim.x * 2) + tid;
    unsigned int mySum = 0;
    if (i < size)
        mySum = d_input[i];
    if (i + blockDim.x < size)
```

```
        mySum += d_input[i + blockDim.x];
    s_data[tid] = mySum;
    __syncthreads();
    for (unsigned int s = blockDim.x / 2; s > 32; s >>= 1) {
        if (tid < s)
            s_data[tid] += s_data[tid + s];
        __syncthreads();
    }
    if (tid == 0)
        d_output[blockIdx.x] = s_data[0];
}

// Product reduction kernel
__global__ void prod_gpu(unsigned int *d_input, unsigned int *d_output, int size)
     {
    extern __shared__ unsigned int s_data[];
    int tid = threadIdx.x;
    int i = blockIdx.x * (blockDim.x * 2) + tid;
    unsigned int myProd = 1; // Identity for product is 1
    if (i < size)
        myProd = d_input[i];
    if (i + blockDim.x < size)
        myProd *= d_input[i + blockDim.x];
    s_data[tid] = myProd;
    __syncthreads();
    for (unsigned int s = blockDim.x / 2; s > 32; s >>= 1) {
        if (tid < s)
            s_data[tid] *= s_data[tid + s];
        __syncthreads();
    }
    if (tid == 0)
        d_output[blockIdx.x] = s_data[0];
}

// Minimum reduction kernel
__global__ void min_gpu(unsigned int *d_input, unsigned int *d_output, int size)
     {
    extern __shared__ unsigned int s_data[];
    int tid = threadIdx.x;
    int i = blockIdx.x * (blockDim.x * 2) + tid;
    unsigned int myMin = UINT_MAX; // Identity for min is UINT_MAX
    if (i < size)
        myMin = d_input[i];
    if (i + blockDim.x < size)
        myMin = min(myMin, d_input[i + blockDim.x]);
    s_data[tid] = myMin;
    __syncthreads();
    for (unsigned int s = blockDim.x / 2; s > 32; s >>= 1) {
        if (tid < s)
            s_data[tid] = min(s_data[tid], s_data[tid + s]);
        __syncthreads();
    }
    if (tid == 0)
        d_output[blockIdx.x] = s_data[0];
}

```

```
// ---------------------
// Main: Timing Comparison
// ---------------------

int main() {
    // Test array sizes from 1 to 100000 in steps of 250.
    int maxArraySize = 100000;
    int step = 500;
    srand(time(NULL));

    printf("ArraySize,SyncTime(ms),AsyncTime(ms)\n");

    // Loop over array sizes
    for (int arraySize = 1; arraySize <= maxArraySize; arraySize += step) {
        float totalSyncTime = 0.0f;
        float totalAsyncTime = 0.0f;
        // Compute number of blocks for the reduction kernels
        int numBlocks = (arraySize + (THREADS_PER_BLOCK * 2 - 1)) / (
            THREADS_PER_BLOCK * 2);

        // Average the execution time over several runs
        for (int iter = 0; iter < AVERAGE; iter++) {
            unsigned int *d_input, *d_output;
            unsigned int *h_array;
            float elapsed;

            // ---------------------------
            // Synchronous Execution Block
            // ---------------------------
            cudaEvent_t start, stop;
            cudaEventCreate(&start);
            cudaEventCreate(&stop);
            cudaEventRecord(start, 0);

            // SUM Operation
            h_array = (unsigned int*)malloc(arraySize * sizeof(unsigned int));
            for (int i = 0; i < arraySize; i++)
                h_array[i] = rand() % 100;
            cudaMalloc((void**)&d_input, arraySize * sizeof(unsigned int));
            cudaMalloc((void**)&d_output, numBlocks * sizeof(unsigned int));
            cudaMemcpy(d_input, h_array, arraySize * sizeof(unsigned int),
                cudaMemcpyHostToDevice);
            sum_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                unsigned int)>>>(d_input, d_output, arraySize);
            cudaDeviceSynchronize();
            free(h_array);
            cudaFree(d_input);
            cudaFree(d_output);

            // PRODUCT Operation
            h_array = (unsigned int*)malloc(arraySize * sizeof(unsigned int));
            for (int i = 0; i < arraySize; i++)
                h_array[i] = (rand() % 9) + 1;
            cudaMalloc((void**)&d_input, arraySize * sizeof(unsigned int));
            cudaMalloc((void**)&d_output, numBlocks * sizeof(unsigned int));
```

```
150             cudaMemcpy(d_input, h_array, arraySize * sizeof(unsigned int),
                    cudaMemcpyHostToDevice);
151             prod_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int)>>>(d_input, d_output, arraySize);
152             cudaDeviceSynchronize();
153             free(h_array);
154             cudaFree(d_input);
155             cudaFree(d_output);
156
157             // MIN Operation
158             h_array = (unsigned int*)malloc(arraySize * sizeof(unsigned int));
159             for (int i = 0; i < arraySize; i++)
160                 h_array[i] = rand() % 100;
161             cudaMalloc((void**)&d_input, arraySize * sizeof(unsigned int));
162             cudaMalloc((void**)&d_output, numBlocks * sizeof(unsigned int));
163             cudaMemcpy(d_input, h_array, arraySize * sizeof(unsigned int),
                    cudaMemcpyHostToDevice);
164             min_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int)>>>(d_input, d_output, arraySize);
165             cudaDeviceSynchronize();
166             free(h_array);
167             cudaFree(d_input);
168             cudaFree(d_output);
169
170             // MAX Operation
171             h_array = (unsigned int*)malloc(arraySize * sizeof(unsigned int));
172             for (int i = 0; i < arraySize; i++)
173                 h_array[i] = rand() % 100;
174             cudaMalloc((void**)&d_input, arraySize * sizeof(unsigned int));
175             cudaMalloc((void**)&d_output, numBlocks * sizeof(unsigned int));
176             cudaMemcpy(d_input, h_array, arraySize * sizeof(unsigned int),
                    cudaMemcpyHostToDevice);
177             max_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int)>>>(d_input, d_output, arraySize);
178             cudaDeviceSynchronize();
179             free(h_array);
180             cudaFree(d_input);
181             cudaFree(d_output);
182
183             cudaEventRecord(stop, 0);
184             cudaEventSynchronize(stop);
185             cudaEventElapsedTime(&elapsed, start, stop);
186             totalSyncTime += elapsed;
187             cudaEventDestroy(start);
188             cudaEventDestroy(stop);
189
190             // ---------------------------
191             // Asynchronous Execution Block (using separate streams)
192             // ---------------------------
193             cudaEventCreate(&start);
194             cudaEventCreate(&stop);
195             cudaEventRecord(start, 0);
196
197             // Create four streams for the four operations
198             cudaStream_t streamSum, streamProd, streamMin, streamMax;
199             cudaStreamCreate(&streamSum);
```

```
200             cudaStreamCreate(&streamProd);
201             cudaStreamCreate(&streamMin);
202             cudaStreamCreate(&streamMax);
203
204             // SUM Operation (async)
205             unsigned int *d_input_sum, *d_output_sum;
206             unsigned int *h_array_sum = (unsigned int*)malloc(arraySize * sizeof(
                    unsigned int));
207             for (int i = 0; i < arraySize; i++)
208                 h_array_sum[i] = rand() % 100;
209             cudaMalloc((void**)&d_input_sum, arraySize * sizeof(unsigned int));
210             cudaMalloc((void**)&d_output_sum, numBlocks * sizeof(unsigned int));
211             cudaMemcpyAsync(d_input_sum, h_array_sum, arraySize * sizeof(unsigned
                    int), cudaMemcpyHostToDevice, streamSum);
212             sum_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int), streamSum>>>(d_input_sum, d_output_sum, arraySize);
213
214             // PRODUCT Operation (async)
215             unsigned int *d_input_prod, *d_output_prod;
216             unsigned int *h_array_prod = (unsigned int*)malloc(arraySize * sizeof(
                    unsigned int));
217             for (int i = 0; i < arraySize; i++)
218                 h_array_prod[i] = (rand() % 9) + 1;
219             cudaMalloc((void**)&d_input_prod, arraySize * sizeof(unsigned int));
220             cudaMalloc((void**)&d_output_prod, numBlocks * sizeof(unsigned int));
221             cudaMemcpyAsync(d_input_prod, h_array_prod, arraySize * sizeof(
                    unsigned int), cudaMemcpyHostToDevice, streamProd);
222             prod_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int), streamProd>>>(d_input_prod, d_output_prod,
                    arraySize);
223
224             // MIN Operation (async)
225             unsigned int *d_input_min, *d_output_min;
226             unsigned int *h_array_min = (unsigned int*)malloc(arraySize * sizeof(
                    unsigned int));
227             for (int i = 0; i < arraySize; i++)
228                 h_array_min[i] = rand() % 100;
229             cudaMalloc((void**)&d_input_min, arraySize * sizeof(unsigned int));
230             cudaMalloc((void**)&d_output_min, numBlocks * sizeof(unsigned int));
231             cudaMemcpyAsync(d_input_min, h_array_min, arraySize * sizeof(unsigned
                    int), cudaMemcpyHostToDevice, streamMin);
232             min_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int), streamMin>>>(d_input_min, d_output_min, arraySize);
233
234             // MAX Operation (async)
235             unsigned int *d_input_max, *d_output_max;
236             unsigned int *h_array_max = (unsigned int*)malloc(arraySize * sizeof(
                    unsigned int));
237             for (int i = 0; i < arraySize; i++)
238                 h_array_max[i] = rand() % 100;
239             cudaMalloc((void**)&d_input_max, arraySize * sizeof(unsigned int));
240             cudaMalloc((void**)&d_output_max, numBlocks * sizeof(unsigned int));
241             cudaMemcpyAsync(d_input_max, h_array_max, arraySize * sizeof(unsigned
                    int), cudaMemcpyHostToDevice, streamMax);
242             max_gpu<<<numBlocks, THREADS_PER_BLOCK, THREADS_PER_BLOCK * sizeof(
                    unsigned int), streamMax>>>(d_input_max, d_output_max, arraySize);
```

```
243
244             // Synchronize all streams
245             cudaStreamSynchronize(streamSum);
246             cudaStreamSynchronize(streamProd);
247             cudaStreamSynchronize(streamMin);
248             cudaStreamSynchronize(streamMax);
249
250             cudaEventRecord(stop, 0);
251             cudaEventSynchronize(stop);
252             cudaEventElapsedTime(&elapsed, start, stop);
253             totalAsyncTime += elapsed;
254             cudaEventDestroy(start);
255             cudaEventDestroy(stop);
256
257             // Destroy the streams
258             cudaStreamDestroy(streamSum);
259             cudaStreamDestroy(streamProd);
260             cudaStreamDestroy(streamMin);
261             cudaStreamDestroy(streamMax);
262
263             // Free asynchronous memory for each operation
264             cudaFree(d_input_sum);
265             cudaFree(d_output_sum);
266             free(h_array_sum);
267
268             cudaFree(d_input_prod);
269             cudaFree(d_output_prod);
270             free(h_array_prod);
271
272             cudaFree(d_input_min);
273             cudaFree(d_output_min);
274             free(h_array_min);
275
276             cudaFree(d_input_max);
277             cudaFree(d_output_max);
278             free(h_array_max);
279         }
280         // Output the averaged execution times for this array size
281         printf("%d, %f, %f\n", arraySize, totalSyncTime / AVERAGE, totalAsyncTime
                / AVERAGE);
282     }
283     return 0;
284 }
```

Listing 2.1: (A)synchronous executions